

2. User-Story Backlog

The university requirements were transformed into simple, generic user stories. Each story was first expressed in a narrative form “As a [user], I want to [do something] so that [result happens],” then organized into the required backlog columns: ID, User, Action, Result / Outcome, and Function. Each story captures one main action and its expected result. The stories focus primarily on user goals and meaningful system behavior. Every story has a unique three-digit ID, and stories belonging to the same feature share the same hundreds digit: 1xx Login, 2xx Register, 3xx Enroll, 4xx Drop, 5xx View, 6xx Change Password, 7xx Admin, and 8xx File I/O. “System” appears in the User column where the action is performed automatically by the application.

2.1. Narrative user stories

1xx — Login

- *As a student, I want to log in with my email and password so that I can access the student subsystem.*
- *As the system, I want to validate the login details so that invalid credentials can be rejected.*

2xx — Register

- *As a visitor, I want to register with my name, email and password so that I become a student who can use the system.*
- *As the system, I want to validate the registration details and assign a unique student ID so that a valid student record can be created.*

3xx — Enroll

- *As a student, I want to enroll in a subject so that it is added to my enrolment list.*
- *As the system, I want to create a subject with a unique ID, random mark and grade so that the enrollment information is complete.*
- *As the system, I want to check the number of enrolled subjects so that a student can't enroll in more than four subjects.*

4xx — Drop

- *As a student, I want to drop a subject so that it is removed from my enrolment list.*

5xx — View

- *As a student, I want to view my enrolment list so that I can see my subjects, marks and grades.*
- *As an admin, I want to view all registered students so that I can see every student on file.*

6xx — Change Password

- *As a student, I want to change my password so that my login credentials stay up to date.*
- *As the system, I want to validate the new password so that only a password meeting the required pattern is accepted.*

7xx — Admin

- *As an admin, I want to view, organize and categorize all students so that I can monitor their results.*
- *As an admin, I want to remove a student or clear all records so that I can organized the student data as I see fit.*

8xx — File I/O

- *As the system, I want to save and load student data so that information persists between sessions.*

2.2 Requirements table (backlog)

ID	User	Action	Result / Outcome	Function
1xx — Login				
100	System	log into the application if given valid credentials	Student is taken to the student subsystem	login()
101	System	match the entered email and password with the ones on file	a registered student is authenticated	match_credentials()
102	Student	Enter credentials	Credential are entered in the login page	enter_credential()
105	System	verify the entered email and password against the required patterns	malformed input is rejected before it is checked against the file	validate_credentials()
106	System	show an error message when the credentials do not match	the student is told the login failed and can retry	show_login_error()
2xx — Register				
201	Student	register by entering a name, an email and a password	a new student account is created	register()
202	System	validate the email against pre-defined patterns (e.g. emails should end with the domain @university.com)	only valid university emails are accepted	validate_email()
203	System	validate the password (upper-case start, at least five letters, followed by three or more digits)	only passwords meeting the criteria are accepted	validate_password()
204	System	generate and assign a unique six-digit student ID (zero-padded, 1–999,999)	each student is uniquely identified	generate_ID()
205	System	show an error message when the Register details are invalid	the visitor is prompted to re-enter valid details	show_register_error()
3xx — Enroll				
301	Student	request to enroll in a subject	the enrolment request is submitted	enrol_subject()
302	System	auto-generate a random mark between 25 and 100 for the enrolled subject	the subject is assigned a mark	generate_mark()
303	System	calculate the subject grade from its mark (Z / P / C / D / HD on the UTS scale)	the subject is assigned a grade	calculate_grade()
304	System	reject enrolment when the list already holds four subjects	the student is notified the four-subject limit is reached	check_subject_limit()

ID	User	Action	Result / Outcome	Function
305	System	generate a unique three-digit ID for the subject (001-999)	a subject ID between 001 and 999 is assigned	generate_subject_ID()
306	System	add the generated subject to the student's enrolment list	the new subject is included in the student's enrolments	add_subject()
4xx — Drop				
401	Student	remove (drop) a subject from the enrolment list	the selected subject is removed from the list	remove_subject()
5xx — View				
501	Student	view the current enrolment list	all enrolled subjects with their marks and grades are displayed	view_enrolment()
502	Admin	view all registered students	every student on file is displayed	view_all_students()
6xx — Change Password				
601	Student	enter the new account password	The new password is submitted for validation	enter_password()
602	System	validate the new password against the required pattern	only a valid new password is accepted	validate_password()
603	System	Update the student's password	The valid new password replaces the previous password	
7xx — Admin				
701	Admin	access the admin subsystem without registering	existing staff can manage students directly	admin_access()
702	Admin	organize and view students grouped by grade	students are displayed grouped by grade	group_by_grade()
703	Admin	categorize students into PASS/FAIL by mark (mark < 50 → FAIL; mark ≥ 50 → PASS)	students are classified by outcome	categorise_pass_fail()
704	Admin	remove an individual student	a single student record is deleted	remove_student()
705	Admin	clear all student records	the students.data file is emptied	clear_all_students()
8xx — File I/O				
801	System	save registered students and their enrolment details to students.data	student data persists between sessions	save_to_file()
802	System	load student data from students.data on start-up	the data is available to the student and admin subsystems	load_from_file()